



Assignment 10: Exploring an Unknown Language through Clustering

handed out: 9 July, 14:00
to be submitted by: 17 July, 22:00

This assignment explores various clustering-based approaches to finding patterns in a corpus from an undocumented language. The example language is Welsh (if you happen to know something about that language, you will have to put yourself into the shoes of someone who does not), and our corpus `cym-os-corpus.txt` is a large collection of random material from the Open Subtitles Corpus (it is made to look like cohesive text in order to increase the mystique, but actually just concatenates various lines from translated subtitles).

Task 1: Loading and Exploring the Data

- Unzip and load the corpus, performing the usual steps for tokenisation: process each line (= paragraph), split it into sentences by the three sentence-final marks (! ? .), then split each sentence into tokens based on spaces, remove potential trailing punctuation (, . :) from each token, and convert it into lowercase. Make sure to avoid (or delete) empty tokens. You should end up with a list of about 5.9 million tokens.
- Use a counter to perform a tally of all characters occurring in the text, and sort the characters by frequency (hint: the most frequent character is `d`, it should occur about 2.5 million times). Compute the logarithms of character probabilities, and store them in a dictionary `log_char_prob`.
- Create a token counter, and extract the 20 most common tokens with their counts.
(Hint: the most common token is `yn`, it occurs around 283000 times)

Task 2: Identifying Vowels and Consonants

When taking a first look at the data, you might have wondered how words which apparently consist only of consonant characters (such as *yw*, *ydyw*, *fyddwn*, *cychwyn*, *synnwyr*) can be articulated. We quickly arrive at the suspicion that some symbols that represent consonants in English actually serve as vowels in Welsh orthography. Clustering symbols by the environments in which they occur can provide us with hints.

- We build a dictionary in which we store for every character the environments in which it is attested. Environments are of shape `A_B` where `A` and `B` are the preceding and following character. In order to represent positions at the start and end of words, we pad each token by the sentinel symbol `#` on both sides. For instance, this means that the token *ydyw* contains attestations of the environments `"#_d"` and `"d_w"` for `"y"`, `"y_y"` for `"d"`, and `"y_#"` for `"w"`.
- Convert the stored attestations into a Pandas dataframe with characters in the rows and environments as columns, where each environment attested for the respective character receives the value 1, and all other environments the value 0. (Hint: reshape the dictionary so that it can be imported via `pd.DataFrame.from_dict`, set all missing entries to 0 after the import). Reduce the dataframe to only those characters which are attested in at least 200 different environments. (Hint: In my solution, the resulting dataframe has 23 rows and 1130 columns of binary values).
- Use agglomerative clustering with the default parameters in order to infer a clustering of the sounds based on the similarity of environments in which they occur. Use the code provided at https://scikit-learn.org/stable/auto_examples/cluster/plot_agglomerative_dendrogram.html to create a dendrogram in order to visualise the clustering. Do the symbols cluster differently from what we would expect in English, and does it seem sensible to reclassify some symbols as vowels?

Task 3: Search for Potential Digraphs

After having established the vowel inventory, we take a closer look at the orthography and get the impression that some sequences of consonants might serve as digraphs (sequences of two orthographic symbols which together represent a single sound), as in English (*sh*, *th*, *wh* etc.). In particular, we suspect that combinations of consonant symbols with *h* and geminates such as *ll* might serve as digraphs. Again, we will try to use clustering in order to see whether the digraphs might fall out as a natural category from their distribution.

a) We start with some intuitions about what might characterise digraphs and distinguish them from consonant clusters. Two ideas seem especially promising and easy to implement: they should be among the most frequent consonant bigrams, and they should not only appear word-internally, but also at the start or end of words. In order to measure these two properties, create counters `cons_bigram_counter` and `internal_bigram_counter` to count the occurrences of bigrams of the relevant shapes (two identical consonants or a consonant followed by *h*) as well as the word-internal occurrences. For both counts, each attested word form is processed only once (token frequency should not factor into the results!).

b) Create a dataframe in which for the 25 most common candidate digraphs, you store the digraph's log probability (suggested column name `jointLogProb`) and the ratio of occurrences which were word-internal (suggested name `internalRatio`). Create a scatter plot in order to understand the structure of the data (drawing text labels onto the node will help you interpret the results later). Good digraphs should be high in joint log probability and low in the word-internal ratio, but without labeled data, we cannot determine directly how these two factors should be weighted. We will attempt to use clustering in order to infer some structure from the distribution within this two-dimensional space.

c) Use k-means clustering in order to divide the space of candidate digraphs into clusters. Use silhouette scores to compare clustering quality for different choices of the number of clusters *k*. There is likely not going to be a clearly preferable number, so you should explore the two best options, and see whether they lead to similar conclusions. Visualise the clustering results using the `hue` attribute of Seaborn's scatter plot function, and use the intuitions to determine the most plausible cluster(s) based on their cluster means. Extract the digraphs contained in the best cluster(s), and compare the results between your two choices of *k*. Compare your conclusions to a list of Welsh digraphs that you can easily find online.

Task 4: Open and Closed Word Classes

All languages have several closed word classes (e.g. prepositions and pronouns) and several open word classes (e.g. nouns and verbs), and we can try to group the words of an unknown language into similar categories based on the intuition that closed word classes typically contain shorter words of high frequency, whereas open word classes mostly consist of longer words of lower frequency.

a) Use the data structures from Task 1 in order to fill a Pandas dataframe with the following two measures for the 5,000 most frequent tokens: 1) log probability (logarithm of the product of character probabilities, can be computed by summing over the values stored in `log_char_prob`) and 2) log frequency (the logarithm of the count in your token counter). Normalise both dimensions (subtract mean and divide by standard deviation), and visualise the data as a scatterplot. Is there any obvious clustering structure?

b) Cluster the data using Gaussian mixture models. Again, work with the default settings of all parameters, but vary the number of components between 1 and 10. Use the Akaike Information Criterion in order to decide on a good number of clusters, and visualise the results using hues in a repetition of the scatter plot.

c) How large are the different clusters? Extract a sample of 20 tokens from each cluster, format them as one token per line, and feed them through Google translate (or similar) to get a rough impression of what kinds of things the tokens from the different clusters mean. Pick the cluster(s) with the highest density of closed word classes among the translations, and look back at your scatter plot to see whether the cluster(s) cover the region of space you would have expected.

Task 5: Detecting Paradigmatic Sets of Markers

As a final step for illustrating the various tasks which can be approached through clustering, we will attempt to determine sets of co-occurring suffixes which might provide some information about inflectional paradigms in a purely data-driven fashion.

- a) Word forms that are in a paradigmatic relationship to each other can be expected to share a longer chunk of symbols which we can tentatively treat as their common stem. We start by indexing our most frequently occurring 5000 forms by their potential stems. Treat any prefix that has at least length 4 as a stem candidate, and use a dictionary to store for each stem candidate the forms which could potentially feature that stem. For instance, my implementation has the set {'meddyg', 'meddygol', 'meddygon', 'meddyliau', 'meddyliol'} stored under 'meddy'.
- b) In the next step, we extract a list of sets of co-occurring suffixes which result from removing the stem from each list. For instance, the example from the previous step would yield the set {'g', 'gol', 'gon', 'liau', 'liol'} as one of the items on our list.
- c) Now we derive overlap counts between each pair of suffixes. Iterate over the sets of co-occurring suffixes, and when processing such a set, increase the overlap count for each pair of suffixes in the set by one. A sparse matrix representation such as nested dictionaries is recommended for this step.
- d) Convert the overlap counts into a Pandas dataframe, and clean it by replacing all missing values by 0. Reduce the matrix on both axes to only those candidate suffixes for which 50 or more overlaps were found, i.e. delete rows or columns which sum up to less than 50.
- e) Apply agglomerative clustering to the resulting symmetric matrix. Since you likely have very many candidate morphs (919 in my implementation), we will not render a dendrogram this time, but only use this as a clustering algorithm which can operate on similarity matrices. Set the distance threshold to 50, and extract the cluster labels inferred for each morph. Print the sets of morphs which were inferred to likely appear as part of the same paradigm, and compare your results to some inflection tables you find online (e.g. <https://www.leerae.co.uk/post/welsh-conjugation-mega-cheatsheet>). To what extent does the procedure appear to have extracted relevant patterns?